

Building Visuals of Infra Architecture

Make your infrastructure **obvious at a glance** with diagrams that show **how it works, why it scales, and how it survives failure**. This guide will help you create 4–6 production-quality diagrams that effectively communicate your architecture.



What Makes a Great Diagram?



1

Purposeful

One clear idea per visual

2

Layered

Hide detail until needed (overview → deep dive)

3

Consistent

Same icons, colors, line styles across all slides

4

Actionable

Labels with owners/SLOs/links where helpful

Your Core Diagram Set (Minimum)

1 System Overview

End-to-end architecture visualization

2 Dataflow / Bronze–Silver–Gold

Data transformation pipeline visualization

3 Serving & Autoscaling Path

API → model → features visualization

4 HA & Failover Topology

Zones/regions resilience structure

5 Observability & SLO Map

Metrics and measurement points

6 Cost & Ownership Map (Optional)

FinOps/RACI visualization

Visual Language & Legend

Shape Conventions

- Rectangles = services
- Cylinders = data stores
- Hexagons = gateways
- Clouds = external services

Line Styles

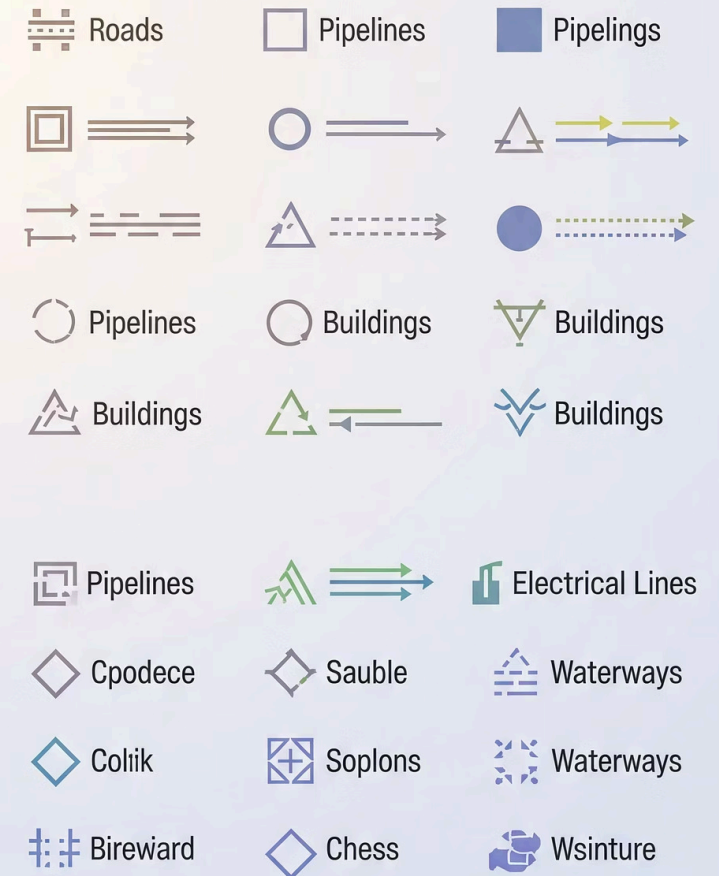
- Solid = synchronous request
- Dashed = asynchronous
- Dotted = control/metrics

Color Coding

- Blue = real-time serving
- Green = batch/ETL
- Orange = observability
- Red outline = failure domain boundary

 **Pro Tip:** Include a small legend box on *every* diagram, even if it seems repetitive.

INFAGMCTURE LEGEND



Tools & Exports



Mermaid

Markdown-friendly, works in docs/CI



PlantUML

Sequence/deployment diagrams



diagrams.net / FigJam

Fast editing with icon libraries



Graphviz

Auto-layout for complex graphs

Export Formats

- **SVG** for slides/docs (scalable)
- **PNG** for README (web-friendly)
- **PDF** for print (high-quality)

Select format based on intended use. SVGs maintain quality at any size but may render differently across platforms.

- 📄 Store source files (.mmd, .puml, .drawio) in version control alongside exports for future updates.

System Overview (Mermaid Template)

flowchart LR

A[Producers] --> | events | K[(Kafka)]

K --> ETL[Airflow ETL] --> S3[(Object Store: Bronze/Silver/Gold)]

S3 --> FE[Feature Store (Offline)]

FE --> TR[Training (DDP/DeepSpeed)] --> REG[Model Registry]

REG --> IMG[(Container Image)]

IMG --> K8S[Inference on Kubernetes]

K8S --> FEON[Feature Store (Online)]

K8S --> MON[(Prometheus/Grafana)]

subgraph HA[HA: Multi-AZ]

K8S

FEON

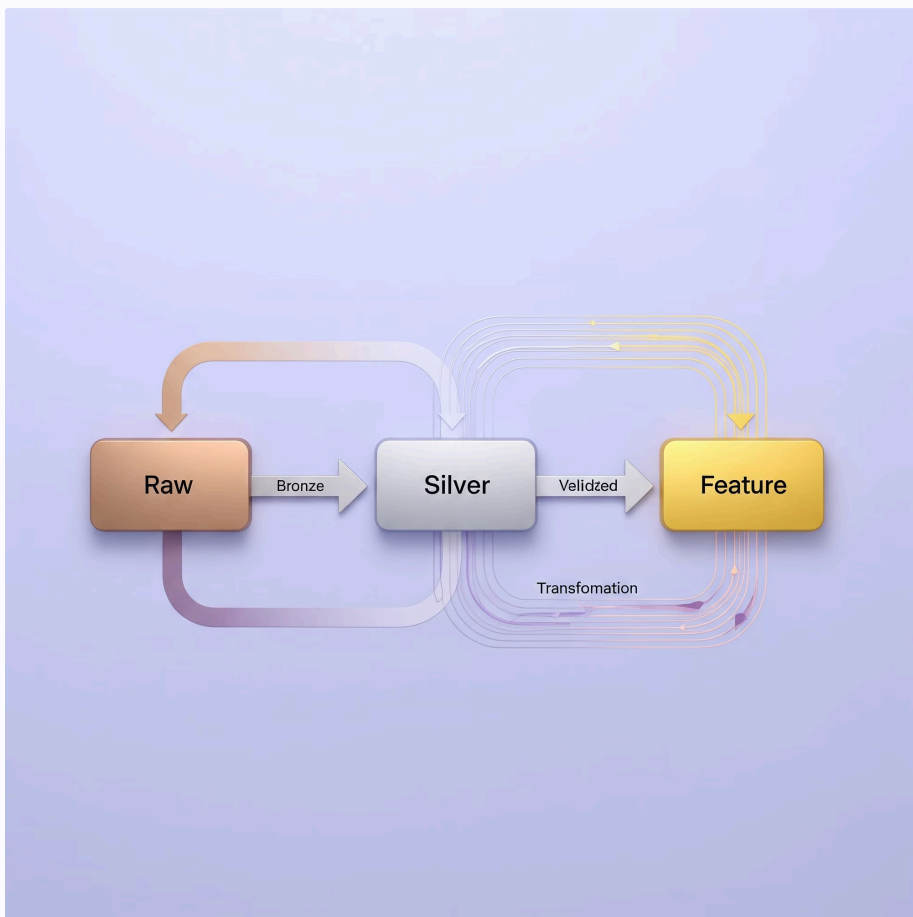
end

Keep text labels concise; annotate **SLOs** next to K8S box (e.g., "p99 < 100ms" or "99.9% availability").

System Overview



Dataflow (Bronze → Silver → Gold)



Bronze (Raw, Append-Only)

- Raw events
- Unvalidated data
- Immutable storage

Silver (Clean, Validated)

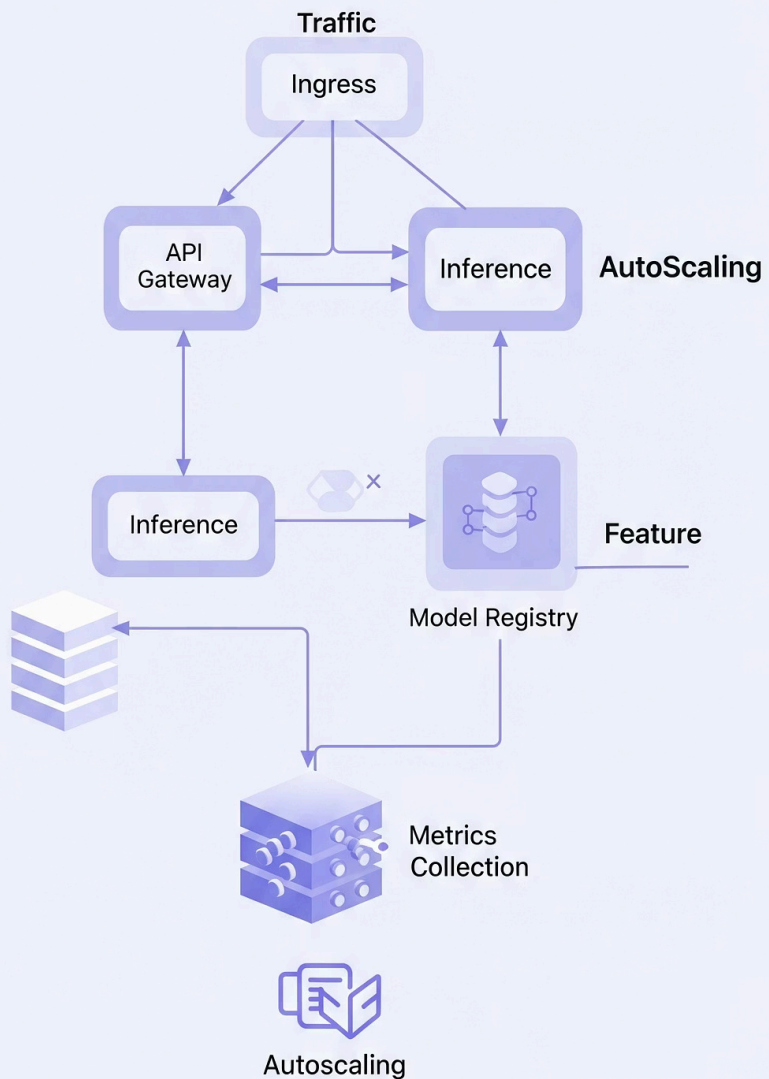
- Validated tables
- Deduped records
- Schema enforced

Gold (Features)

- Feature tables
- Joined data
- ML-ready format

❏ Add **Great Expectations** checks on edges between tiers for data quality assurance.

Machine Learning Serving Architecture



Serving Path & Autoscaling (Component View)

flowchart LR

ING[Ingress/WAF] --> GW[API Gateway]

GW --> SVC[Inference Service (FastAPI/Triton)]

SVC --> FS[(Online Feature Store)]

SVC --> REG[Model Registry]

SVC --> MET[(Prometheus / /metrics)]

subgraph K8s

SVC-.HPA metrics.->HPA[Horizontal Pod Autoscaler]

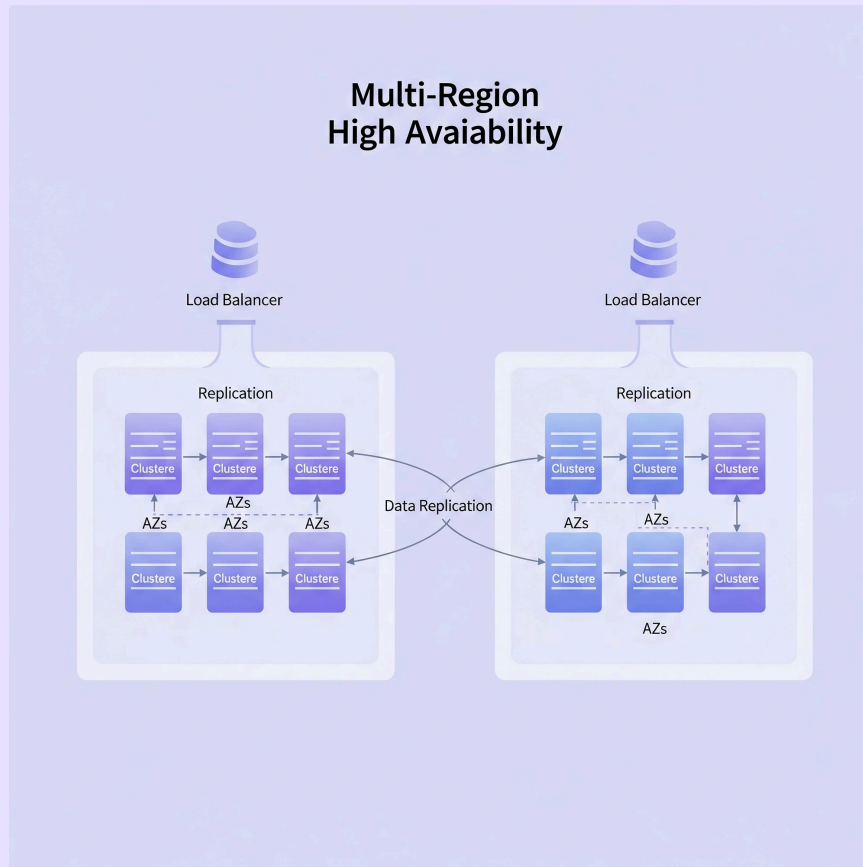
end

GW-.mTLS.->SVC

Add callouts for:

- p95 latency SLO (e.g., "p95 < 300ms")
- HPA scaling rule (e.g., "CPU > 70%, max 12 pods")
- Request budget/token limits (e.g., "100 req/s per client")

HA & Failover (Regions/Zones)



Region Design

- Multiple AZs per region (a/b)
- Region-level load balancing
- Cross-region replication (CRR)

Configuration Options

Clearly indicate:

- **Active-Active** or **Active-Passive**
- **RTO**: Recovery Time Objective (e.g., < 5 min)
- **RPO**: Recovery Point Objective (e.g., < 30 sec)

⚠ Highlight failure domain boundaries with **red outlines** to clarify isolation.

Observability & SLO Map

API Metrics

- Request Duration Histogram → $p95 < 300ms$
- Error Rate → $5xx < 0.1\%$
- Request Volume → Capacity planning

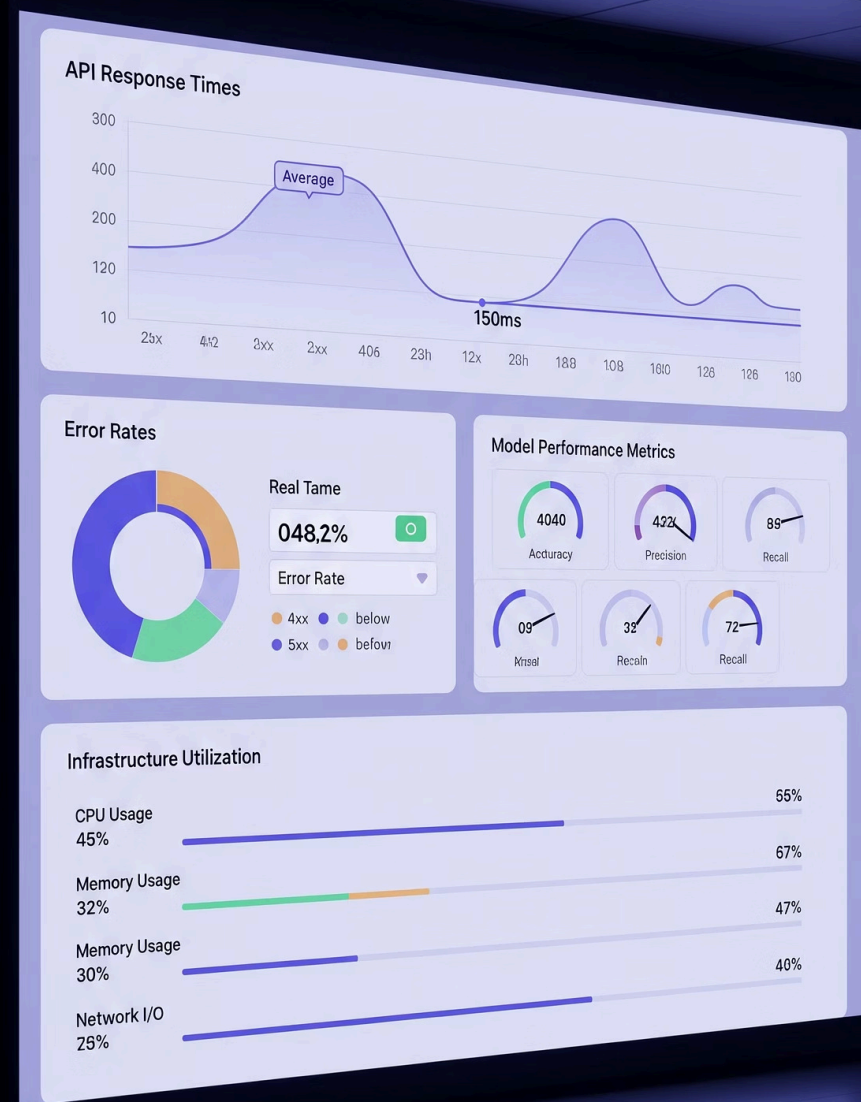
Model Metrics

- Win-rate vs baseline → QoS stable
- Drift PSI → $PSI < 0.25$
- Prediction latency → Performance budgeting

Infrastructure Metrics

- GPU Utilization % → $>60\%$ avg
- Memory consumption → Rightsizing
- Pod restart count → Stability tracking

Link each SLO tile to the corresponding **Grafana panel** in your deck notes for quick access during incidents.

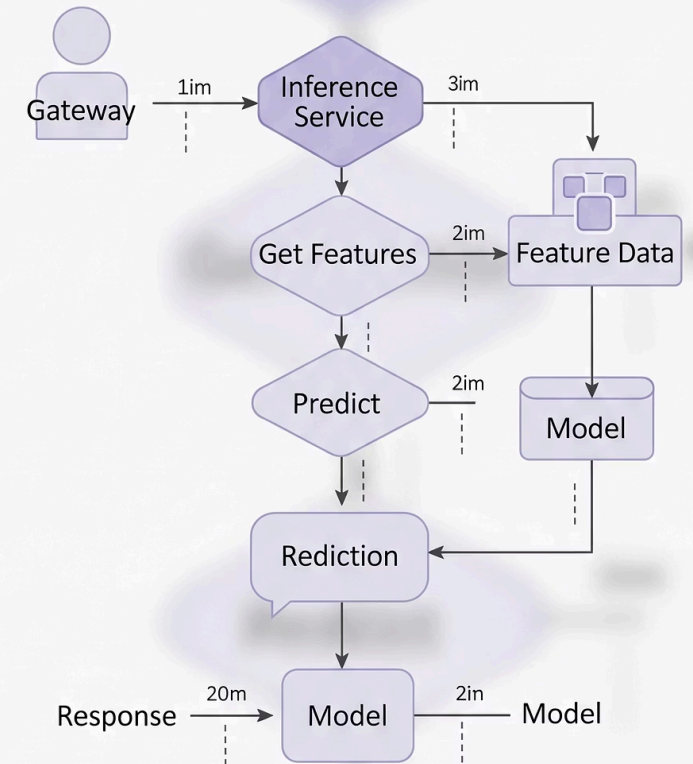


Sequence Diagram (Troubleshooting Narrative)

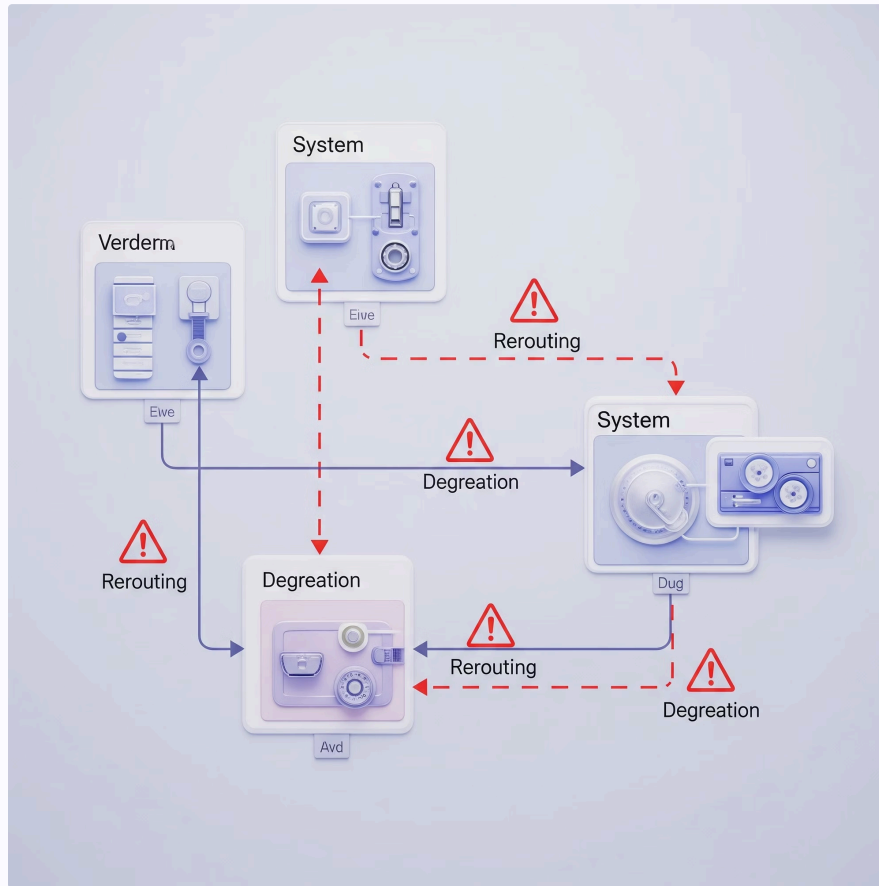
Use PlantUML for request lifecycle & latency attribution:

```
@startuml
actor Client
Client -> Gateway: POST /predict
Gateway -> Inference: forward (JWT, trace_id)
Inference -> FeatureStore: read features
FeatureStore --> Inference: features
Inference -> Model: infer()
Model --> Inference: result (ms)
Inference --> Gateway: 200 + latency_ms
Gateway --> Client: response
@enduml
```

i Add **alt-blocks** for **cache miss** vs **cache hit** scenarios if relevant to your architecture.



Failure Mode Overlay



Common Failure Scenarios

Node Failure

Deployment reschedules (PDB prevents 0 replicas)

Feature Store Degradation

Degraded mode (fallback defaults + retry pattern)

Canary Deployment Failure

[Automatic rollback](#) via GitOps controller

Use dashed **red paths** and warning icons to highlight failure routing.

Security & IAM Map

Edge Security

- OAuth2/OIDC (users)
- API gateway authorization
- WAF protection

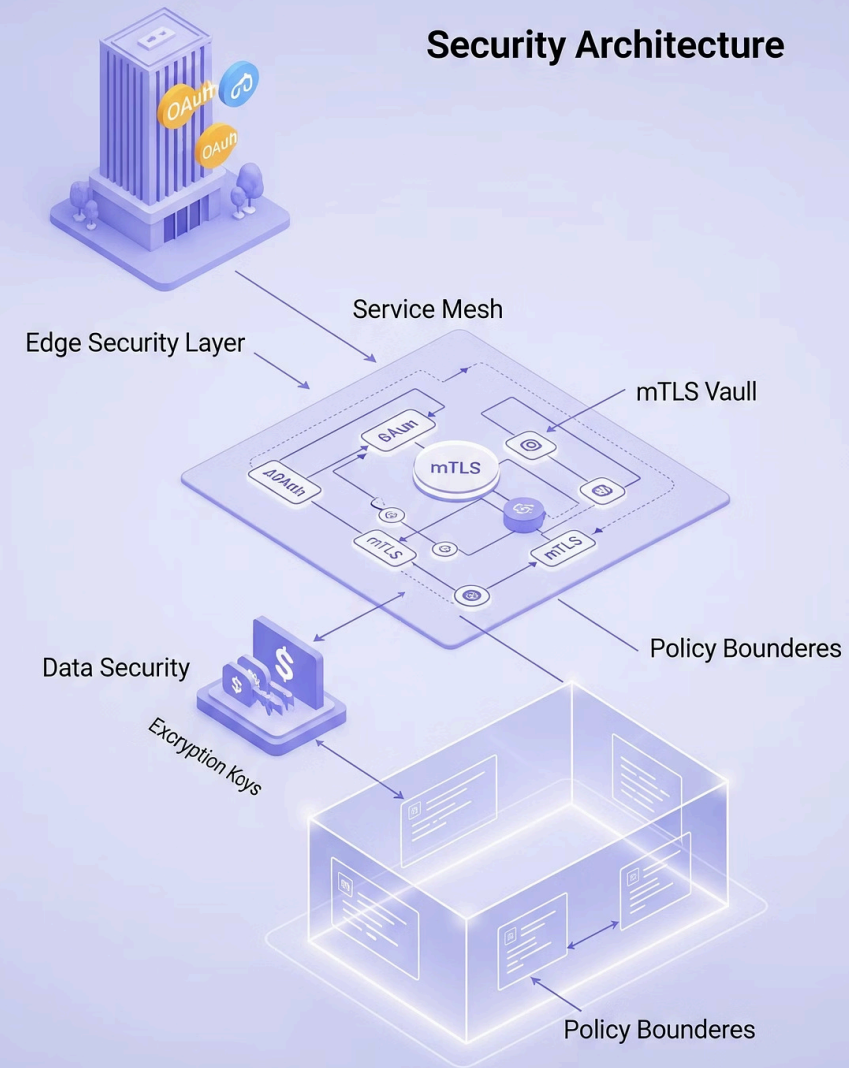
Service Mesh

- mTLS + SPIFFE (services)
- Service-to-service auth
- Traffic encryption

Data Security

- KMS keys
- Secrets via External Secrets
- Encryption at rest

Show **policy boundaries**: Kubernetes namespaces, NetworkPolicies, and RBAC. Use subtle lock icons only where **enforcement happens**.



Cost & Ownership Swimlane (RACI)

Platform Team - K8s nodes, Ingress, HPA - Cost: \$ compute/hr - Budget: \$25K/month - Owner: Platform Lead	ML Engineering - Model image, feature store usage - Cost: \$ /1k requests - Budget: \$15K/month - Owner: ML Ops Lead
Data Engineering - ETL + storage tiers - Cost: \$ /TB-month - Budget: \$20K/month - Owner: Data Platform Lead	

Add monthly budget targets right on the swimlanes for transparency and FinOps visibility.

Swinn Name

Team Name		Owners	
Key Components	250 Metrics	Cost Metrics	1600
Fesoileations	120 Metrics	Owners	

Team Name		Owners	
Key Components	280 Metrics	Cost Metrics	1300
Cost Metrics	120 Metrics	9 Owners	

Team Name		Owners	
Key Components	380 Metrics	Cost Metrics	1200
Papaics	120 Metrics	3 Owners	1600

Team Name		Owners	
Key Components	250 Metrics	Cost Metrics	1200
Feromes	150 Metrics	9 Owners	

Storyboarding the Presentation



Presentation Flow

Start **macro** → **micro**:

1. System overview
2. Serving path
3. HA & failover
4. Observability

For each diagram:

- Begin with 1 sentence "what's the point?"
- Use **progressive reveal**: gray-out non-relevant parts as you zoom
- Keep a **backup slide** with full labels for reference

Accessibility & Clarity

1

Text Size

Minimum 14–16 pt labels on exported images

2

Color Contrast

High-contrast colors; don't rely on color alone (use line styles/symbols)

3

Alternative Text

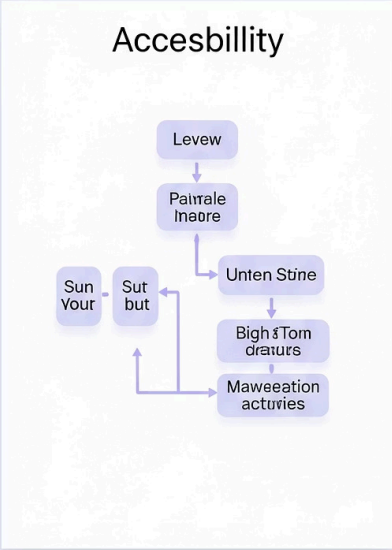
Add [alt text](#) in docs and [captions](#) in slides

4

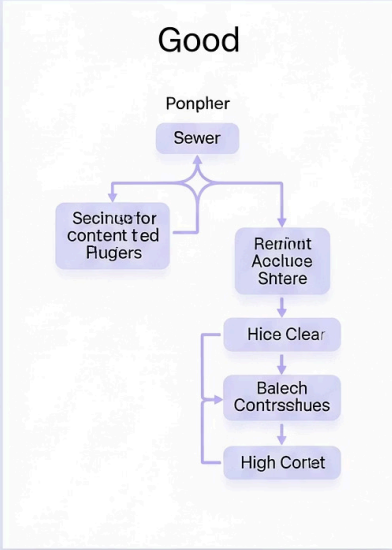
Layout Clarity

Keep arrows orthogonal; avoid crossed lines if possible

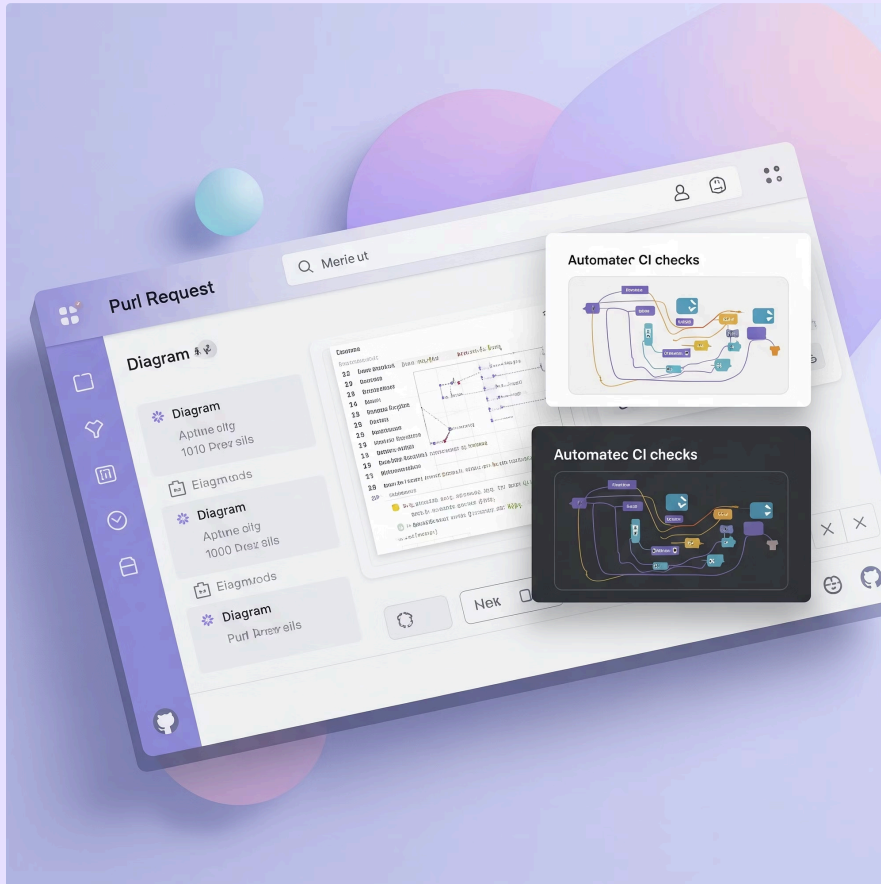
Poor



Excellent Accessibility



Versioning & CI for Diagrams



Version Control Best Practices

- Store source files in `docs/diagrams/`
 - `.mmd` (Mermaid)
 - `.puml` (PlantUML)
 - `.drawio` (diagrams.net)
- CI step renders to **SVG/PNG** on push
- Stamp **commit SHA + date** in footer
- Review diagrams via PR like code

i This approach treats diagrams as code, enabling proper change history tracking and review processes.

Export Pack (What to Hand In)

Required Diagrams (SVG format)

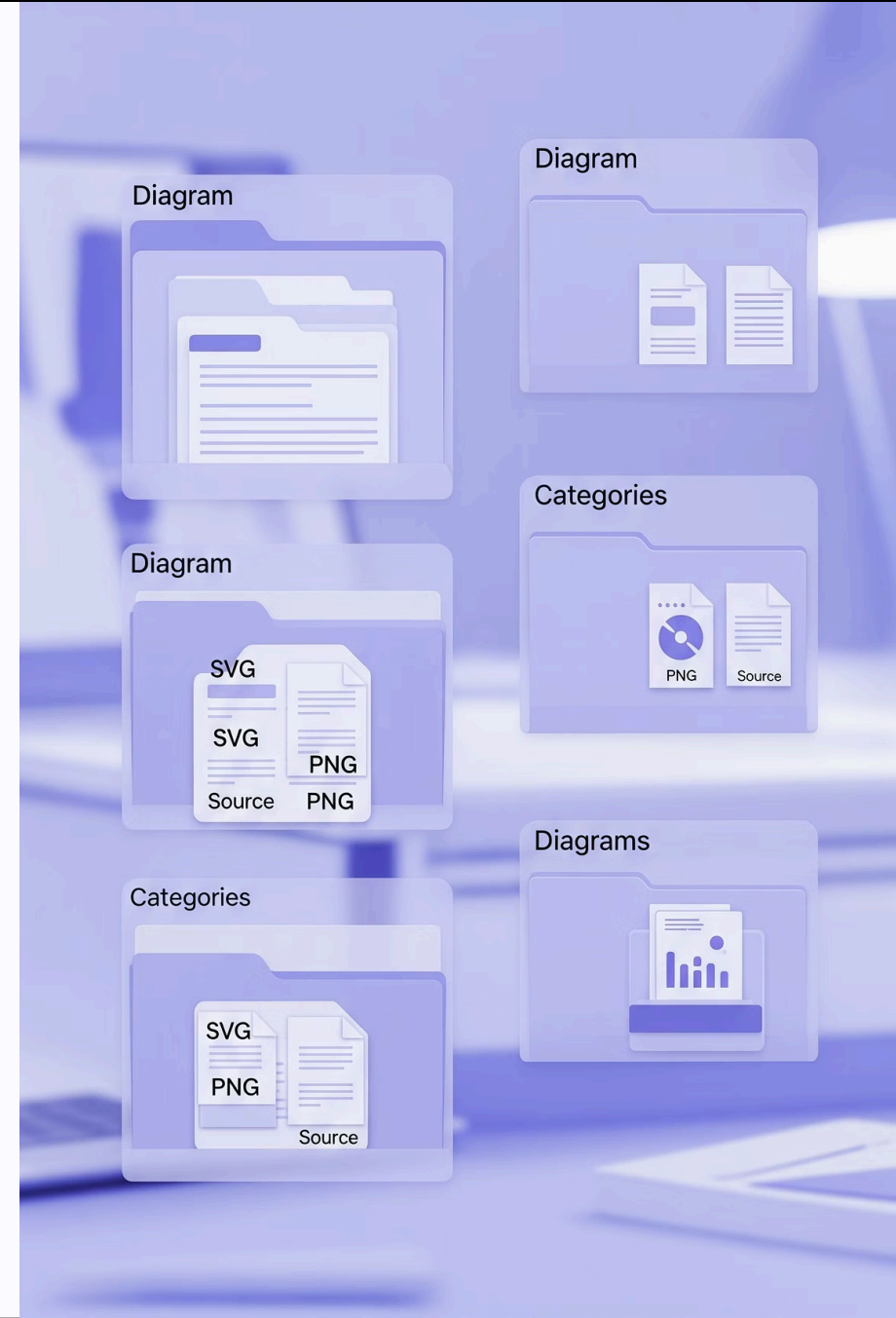
- 01-overview.svg
- 02-dataflow.svg
- 03-serving-autoscale.svg
- 04-ha-failover.svg
- 05-observability-slo.svg

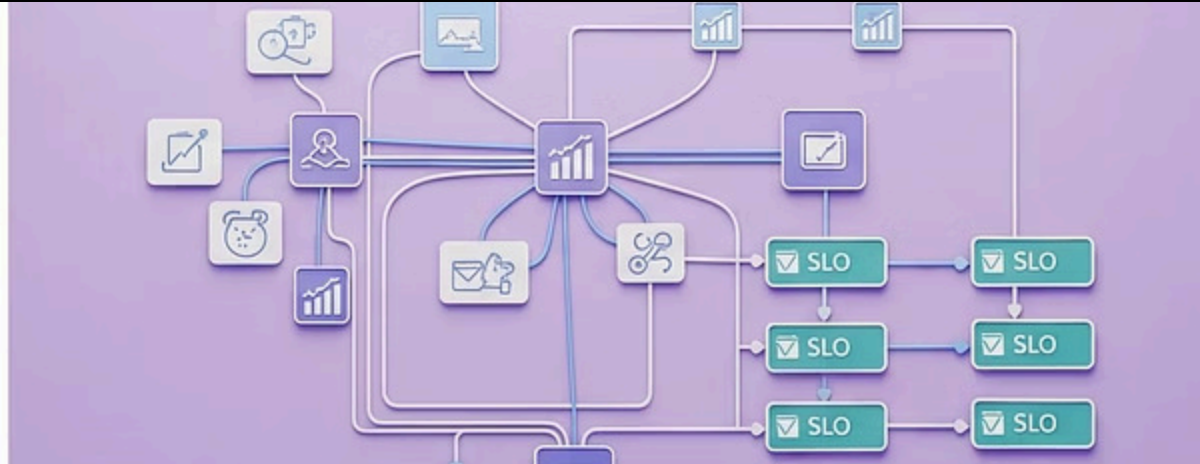
Optional Additions

- 06-security-iam.svg
- 07-cost-raci.svg

Supporting Materials

- A **1-page legend** explaining all symbols
- A **link index** to dashboards/runbooks





Before/After Example (Polish Tips)

Before

- Dense, overwhelming diagram
- Inconsistent icons
- Unlabeled lines
- No clear message
- Difficult to understand at a glance

After

- 6–8 clear, focused boxes
- Consistent legend
- Named edges ("gRPC", "Prometheus scrape")
- [SLO badges](#) on key boxes
- Owner tags for accountability

Checklist (Copy/Paste)

Focus & Consistency

One idea per diagram; consistent legend/colors

Annotations

SLOs & owners annotated where relevant

Boundaries

HA boundaries and failure domains marked

Security

Security controls shown at enforcement points

Exports

Exported as SVG + small PNG; sources tracked in repo

Documentation

Linked in README and final report (Day 356)

Summary



Key Takeaways

- Diagrams translate complexity into **clarity**
- Layer your visuals, annotate SLOs and ownership
- Standardize style; export and version in CI
- Your visuals should **stand alone** without narration

Great architecture diagrams don't just document — they *communicate* and *convince*.

Discussion Prompt

Which diagram will have the biggest impact?

Which diagram will have the biggest impact in your demo:

Overview, Serving Path, or HA & Failover

...and what's the one message you want it to deliver?



Bonus: Tiny CI Snippet (render Mermaid to SVG)

```
name: render-diagrams
on: [push]
jobs:
  mermaid:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm i -g @mermaid-js/mermaid-cli
      - run: |
          mkdir -p docs/diagrams/out
          for f in docs/diagrams/*.mmd; do
            mmdc -i "$f" -o "docs/diagrams/out/${basename "$f%.mmd"}.svg"
          done
```

This CI workflow automatically renders your Mermaid diagrams to SVG files whenever you push changes, ensuring documentation stays in sync with your infrastructure design.

Continuous integration

